

- Chapters 1-4 considered supervised learning for deep neural nets.
 - ↳ These networks are fully connected, with just 1 path from input $\rightarrow$ output
 - ↳ The next chapters introduce more specialized networks.
- This chapter describes **convolutional layers**.
 - ↳ Used mainly for images.
- Images have 3 properties that suggest they need specialized model architecture.
 - 1) **They're high dimensional.**
 - Typical image for classification is 224×224 RGB values. [150K input dimensions]
 - Hidden layers are usually larger than the input layer for a fully connected network means there are at least $150K^2 \approx 22$ billion weights.
 - Poses a problem in terms of required training data & memory.
 - 2) **Nearby image pixels are related.**
 - Fully connected networks have no notion of "nearby", $\therefore$ treat the relationship between every input equally.
 - 3) **The interpretation of an image is stable under geometric transformations.**
 - An image of a tree is still an image of a tree if we shift it left a few pixels.
 - But this shift changes every input of a fully-connected network.
 - So it must learn the patterns of pixels that signify a tree separately at every position. **Very inefficient.**
- Convolutional layers process each local image region independently, using parameters shared across the image.
 - ↳ They need fewer parameters than fully connected networks, $\therefore$ exploit the spatial relationship between nearby pixels.
 - ↳ Don't need to relearn the interpretation of the pixels at every position.

10.1: Invariance & Equivariance:

- Some properties of an image are stable under transformations.
- A function $f(x)$ of an image x is **invariant** to a transformation $t(x)$ if:
$$f[t(x)] = f(x)$$
 - * The output of the function $f(x)$ is the same regardless of the transformation.
- Image classification should be invariant to transformations.
 - ↳ The output should be the same despite:
 - Rotations
 - Translations
 - Flipped
 - Warped
 - Etc.
- A function $f(x)$ of an image x is **equivariant** to a transformation $t(x)$ if its output changes in the same way under the transformation as the input:
$$f[t(x)] = t[f(x)]$$

$$f[\tau[\tau x]] = \tau[f[\tau x]]$$

- Image Segmentation Should be equivariant to transformations.

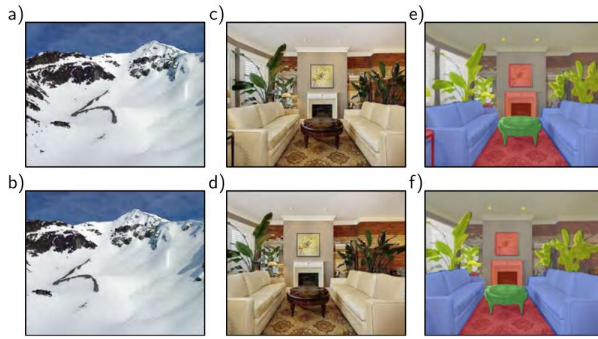


Figure 10.1 Invariance and equivariance for translation. a-b) In image classification, the goal is to categorize both images as “mountain” regardless of the horizontal shift that has occurred. In other words, we require the network prediction to be invariant to translation. c,e) The goal of semantic segmentation is to associate a label with each pixel. d,f) When the input image is translated, we want the output (colored overlay) to translate in the same way. In other words, we require the output to be equivariant with respect to translation. Panels c-f) adapted from Boussemham et al. (2021).

10.2: Convolutional Networks for 1D Inputs:

- CNNs consist of a series of convolutional layers, each of which is equivariant to translation.
↳ Usually include Pooling Mechanisms which induce Partial invariance to translation.

1D Convolutional Operation:

- Convolutional layers are based on the **Convolution** operation.
- In 1D, a convolution transforms input vector x to output vector z so that each output z_i is a weighted sum of nearby inputs.
- The same weights are used at every position; are collectively called the Convolutional Kernel / Filter.
- The size of the region over which inputs are combined is the **Kernel size**.
- For a kernel of size 3, we have:

$$z_i = w_1 x_{i-1} + w_2 x_i + w_3 x_{i+1}$$

where $\overline{w} = [w_1, w_2, w_3]^T$ is the kernel

↳ Note that convolution is equivariant with respect to translation.

↳ If we translate x , z is translated in the same way.

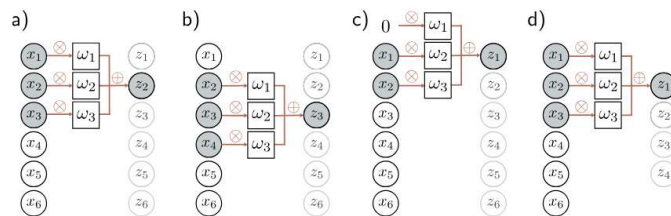


Figure 10.2 1D convolution with kernel size three. Each output z_i is a weighted sum of the nearest three inputs x_{i-1} , x_i , and x_{i+1} , where the weights are $\omega = [\omega_1, \omega_2, \omega_3]$. a) Output z_2 is computed as $z_2 = \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_3$. b) Output z_3 is computed as $z_3 = \omega_1 x_2 + \omega_2 x_3 + \omega_3 x_4$. c) At position z_4 , the kernel extends beyond the first input x_1 . This can be handled by zero-padding, in which we assume values outside the input are zero. The final output is treated similarly. d) Alternatively, we could only compute outputs where the kernel fits within the input range ("valid" convolution); now, the output will be smaller than the input.

Padding:

- There are 2 approaches when dealing with the beginning ; ending of the inputs, where there are no subsequent/previous inputs:

1) Pad the edges of the input ; proceed as normal.

↳ Zero-padding assumes the input is 0 outside of its valid range.

2) Discard the output positions where the kernel exceeds the range of input positions.

↳ "Valid convolutions" have the advantage of introducing no extra information to the edges of the input.

↳ Disadvantage: The representation decreases in size.

Stride, Kernel Size, ; Dilation:

- Convolution operations can differ in **Stride, Kernel Size, ; dilation rate**.
- When we evaluate the output at every position, the **stride = 1**.
 - ↳ It's possible to shift the kernel by a stride > 1 .
 - ↳ If stride = 2, we create ~ half the number of outputs.
- The kernel size can be increased.
 - ↳ We typically keep it odd, so it can be centered around the current position.
 - ↳ Increasing kernel size has the disadvantage of requiring more weights.
- Leads to the idea of **dilated convolutions**.
 - ↳ Kernel values are interspersed with 0s.
 - ↳ Ex: We can turn a kernel of size 5 into a kernel of size 3 by setting the second ; fourth elements to 0. We still integrate information from a larger input region, but only need 3 weights to do this.
- ↳ The number of 0s we intersperse between weights

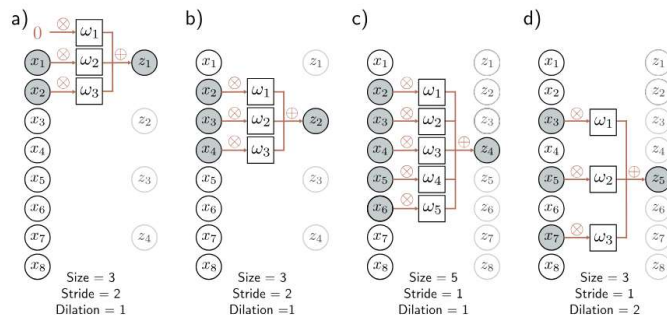


Figure 10.3 Stride, kernel size, and dilation. a) With a stride of two, we evaluate the kernel at every other position, so the first output z_1 is computed from a weighted sum centered at x_1 , and b) the second output z_2 is computed from a weighted sum centered at x_3 and so on. c) The kernel size can also be changed. With a kernel size of five, we take a weighted sum of the nearest five inputs. d) In dilated or atrous convolution (from the French “à trous” – with holes), we intersperse zeros in the weight vector to allow us to combine information over a large area using fewer weights.

Convolutional Layers:

- A convolutional layer computes its output by convolving the input, adding a bias B , & passing each result through an activation function $a[\cdot]$.
- With Kernel Size = 3, Stride = 1, & dilation rate = 1 the i^{th} hidden unit h_i would be computed as:

$$h_i = a[B + w_1 x_{i-1} + w_2 x_i + w_3 x_{i+1}]$$

$$= a[B + \sum_{j=1}^3 w_j x_{i+j-2}]$$

↳ Where B, w_1, w_2, w_3 are trainable parameters.

↳ This is a special case of a fully connected layer that computes the i^{th} hidden unit as:

$$h_i = a[B_i + \sum_{j=1}^3 w_{ij} x_j]$$

- If there are D inputs x , & D hidden units h , this fully connected layer would have D^2 weights & D biases.
- The convolutional layer only uses 3 weights & 1 bias.
- A fully connected network can reproduce this if most of the weights are set to 0, & others are constrained to be identical.

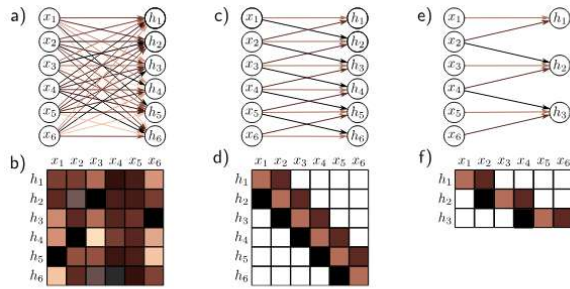


Figure 10.4 Fully connected vs. convolutional layers. a) A fully connected layer has a weight connecting each input x to each hidden unit h (colored arrows) and a bias for each hidden unit (not shown). b) Hence, the associated weight matrix Ω contains 36 weights relating the six inputs to the six hidden units. c) A convolutional layer with kernel size three computes each hidden unit as the same weighted sum of the three neighboring inputs (arrows) plus a bias (not shown). d) The weight matrix is a special case of the fully connected matrix where many weights are zero and others are repeated (same colors indicate same value, white indicates zero weight). e) A convolutional layer with kernel size three and stride two computes a weighted sum at every other position. f) This is also a special case of a fully connected network with a different sparse weight structure.

Channels:

- If we only apply a single convolution, information will likely be lost.
 - ↳ Due to averaging nearby inputs, & ReLU Clipping.
- So we compute several convolutions in parallel.
 - ↳ Each convolution produces a new set of hidden variables.
 - ↳ Called a feature set / channel

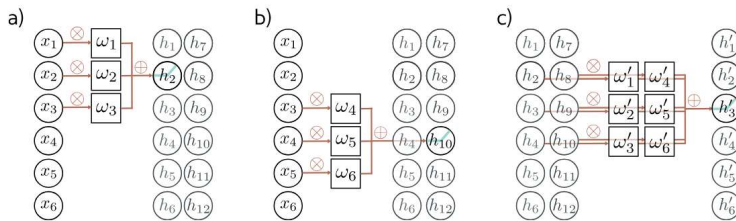


Figure 10.5 Channels. Typically, multiple convolutions are applied to the input x and stored in channels. a) A convolution is applied to create hidden units h_1 to h_6 , which form the first channel. b) A second convolution operation is applied to create hidden units h_7 to h_{12} , which form the second channel. The channels are stored in a 2D array H_1 that contains all the hidden units in the first hidden layer. c) If we add a further convolutional layer, there are now two channels at each input position. Here, the 1D convolution defines a weighted sum over both input channels at the three closest positions to create each new output channel.

- Generally, the input & hidden layers all have multiple channels.
 - ↳ If the incoming layer has C_i channels, & we select a kernel size K per channel, then the hidden units in each output channel is computed as a weighted sum over all C_i channels & K kernel entries using a weight matrix $\tilde{W} \in \mathbb{R}^{C_i \times K}$ & 1 bias.
 - ↳ So if there are C_o channels in the next layer, then we need $\tilde{W} \in \mathbb{R}^{C_i \times C_o \times K}$ weights & $B \in \mathbb{R}^{C_o}$ biases.

Convolutional Networks & Receptive Fields:

- Convolutional networks are comprised of a series of convolutional layers.
- The receptive field of a hidden unit in the network is the region of the original input that feeds into it.

• Consider:

↳ Convolutional network, each convolutional layer has Kernel size = 3

↳ The hidden units of the first layer take a weighted sum of the three closest inputs

↳ So receptive field of size 3.

↳ The units in the second layer take a weighted sum of the 3 closest positions in the first layer.

↳ The positions in the 1st layer are themselves the weighted sums of 3 inputs.

↳ So the second layer has a receptive field of size 5.

• The receptive field of units in successive layers increases, ∴ information from across the input is gradually integrated.

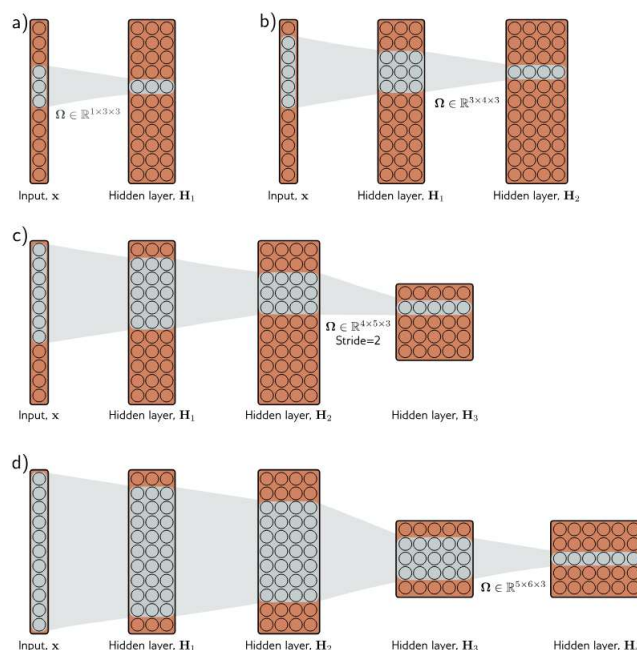


Figure 10.6 Receptive fields for network with kernel width of three. a) An input with eleven dimensions feeds into a hidden layer with three channels and convolution kernel of size three. The pre-activations of the three highlighted hidden units in the first hidden layer H_1 are different weighted sums of the nearest three inputs, so the receptive field in H_1 has size three. b) The pre-activations of the four highlighted hidden units in layer H_2 each take a weighted sum of the three channels in layer H_1 at each of the three nearest positions. Each hidden unit in layer H_1 weights the nearest three input positions. Hence, hidden units in H_2 have a receptive field size of five. c) The hidden units in the third layer (kernel size three, stride two) increases the receptive field size to seven. d) By the time we add a fourth layer, the receptive field of the hidden units at position three have a receptive field that covers the entire input.

Example: MNIST-1D:

• The input x is a 40 D vector.

• The output f is a 10 D vector.

↳ When Passed through Softmax, Produces class Probabilities.

• We use a network with 3 hidden layers.

↳ The 15 channels of H_1 are computed using Kernel size 3, stride = 2 w/ "Valid" padding.

↳ Gives 11 Spatial Positions.

↳ H_2 also has $K=3$, Stride=2, "Valid Padding".

↳ So does H_3

↳ At this stage, the representation has 4 Spatial Positions, 15 Channels.

↳ At this stage, the representation has 4 spatial positions, 15 channels.

↳ These values are reshaped into a vector of size 60, which is mapped by a fully connected layer to the 10 output activations.

• The network was trained for 100,000 steps using SGD w/o momentum, LR=0.01, batch size of 100 on a dataset of 4000 examples

• Compared to a fully connected NN w/ the same # of layers & hidden units [3 layers, 205, 155, 60 units respectively]

↳ The Conv net has 2,050 Parameters vs. the fully connected NN, 59,065 Parameters.

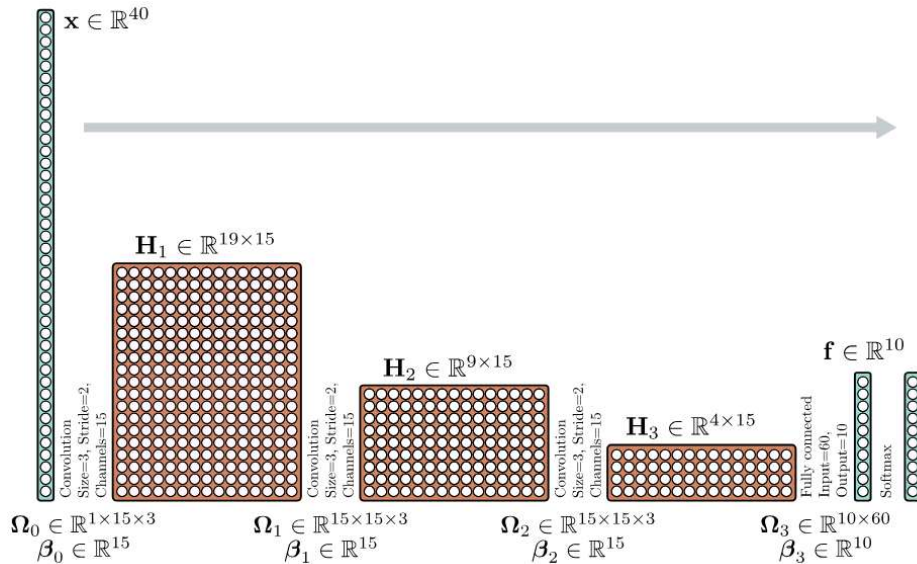


Figure 10.7 Convolutional network for classifying MNIST-1D data (see figure 8.1). The MNIST-1D input has dimension $D_i = 40$. The first convolutional layer has fifteen channels, kernel size three, stride two, and only retains “valid” positions to make a hidden layer with nineteen positions and fifteen channels. The following two convolutional layers have the same settings, gradually reducing the representation size at each subsequent hidden layer. Finally, a fully connected layer takes all sixty hidden units from the third hidden layer. It outputs ten activations that are subsequently passed through a softmax layer to produce the ten class probabilities.

• Despite being a special case of a fully connected network, it fits the training data perfectly, & has much less test error.

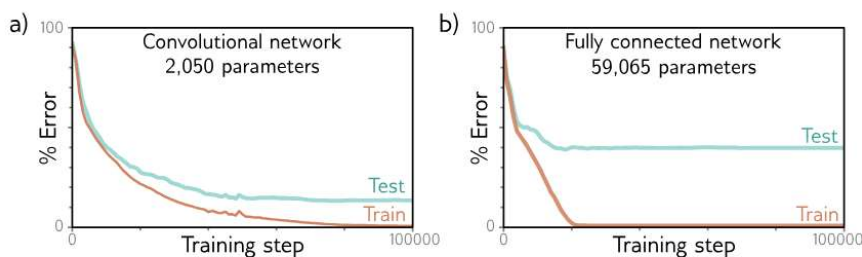


Figure 10.8 MNIST-1D results. a) The convolutional network from figure 10.7 eventually fits the training data perfectly and has ~17% test error. b) A fully connected network with the same number of hidden layers and the number of hidden units in each learns the training data faster but fails to generalize well with ~40% test error. The latter model can reproduce the convolutional model but fails to do so. The convolutional structure restricts the possible mappings to those that process every position similarly, and this restriction improves performance.

~40% test error. The latter model can reproduce the convolutional model but fails to do so. The convolutional structure restricts the possible mappings to those that process every position similarly, and this restriction improves performance.

- This discrepancy is probably not due to the difference in parameters.

↳ Overparametrization usually improves performance.

- The likely explanation is that the architecture has a superior inductive bias.

↳ Because we have embodied some prior knowledge into the architecture.

↳ Told the network to process each position in the input in the same way.

- The fully connected network has to learn what each digit looks like at every position.

↳ The CNN shares information across positions ; thus learns to identify each category more accurately.

- Other ways of looking at it:

↳ When training, we search through a family of smaller family of input/output mappings, all of which are plausible.

↳ The convolutional structure can be considered a regularizer that applies an infinite penalty to most of the solutions that a fully connected NN can describe.

10.3: Convolutional Networks for 2D Inputs:

- Conv nets for 1D inputs can be used for:

↳ Financial time series

↳ Audio

↳ Text

- CNNs are usually applied to 2D image data

↳ The kernel is a 2D object

↳ A 3×3 kernel $\mathcal{K} \in \mathbb{R}^{3 \times 3}$ applied to a 2D input w/ elements x_{ij} computes a single layer of hidden units h_{ij} as:

$$h_{ij} = a \left[b + \sum_{m=1}^3 \sum_{n=1}^3 w_{mn} x_{i+m-2, j+n-2} \right]$$

↳ Entries of the convolutional kernel.

* Just a weighted sum over a square 3×3 region

- The kernel is translated both horizontally ; vertically across the 2D input to create an output at each position.

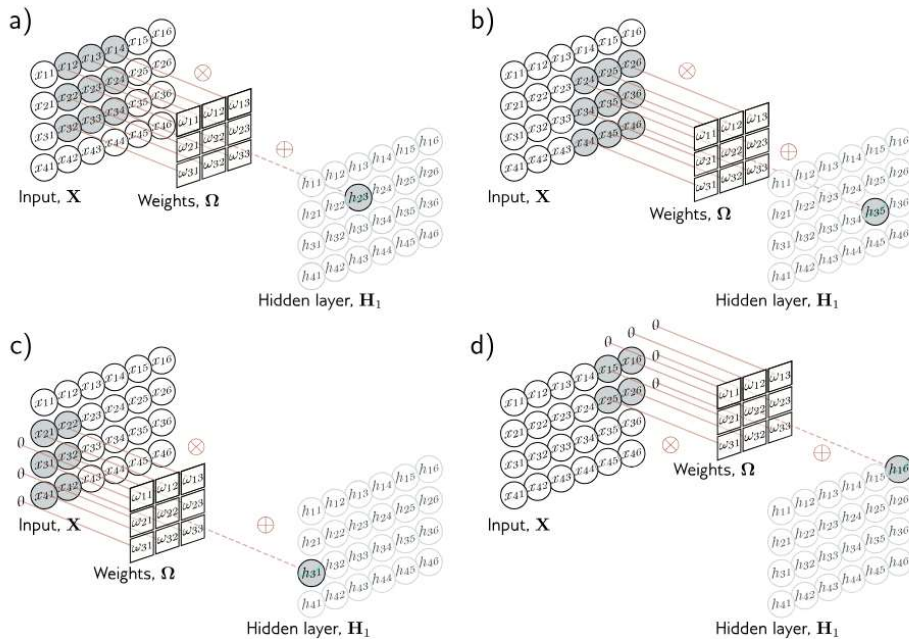


Figure 10.9 2D convolutional layer. Each output h_{ij} computes a weighted sum of the 3×3 nearest inputs, adds a bias, and passes the result through an activation function. a) Here, the output h_{23} (shaded output) is a weighted sum of the nine positions from x_{12} to x_{34} (shaded inputs). b) Different outputs are computed by translating the kernel across the image grid in two dimensions. c–d) With zero-padding, positions beyond the image's edge are considered to be zero.

- Usually the input is an RGB image.

- ↳ Treat it as a 2D signal w/ 3 channels.

- ↳ A 3×3 kernel would have $3 \times 3 \times 3$ weights

- ↳ Create a 2D output with the same height/width of the input image [assuming zero padding]

- ↳ For multiple channels, repeat this process w/ different kernel weights ; append to form a 3D tensor.

- ↳ If kernel size is $K \times K$, ; there are C_i input channels, each output channel is a weighted sum of $C_i \times K \times K$ quantities + a bias.

- ↳ To compute C_o output channels, we need $C_i \times C_o \times K \times K$ weights ; C_o biases.

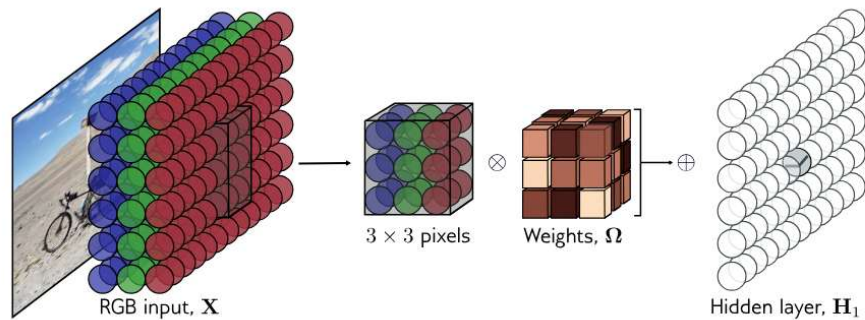


Figure 10.10 2D convolution applied to an image. The image is treated as a 2D input with three channels corresponding to the red, green, and blue components. With a 3×3 kernel, each pre-activation in the first hidden layer is computed by pointwise multiplying the $3 \times 3 \times 3$ kernel weights with the 3×3 RGB image patch centered at the same position, summing, and adding the bias. To calculate all the pre-activations in the hidden layer, we “slide” the kernel over the image in both horizontal and vertical directions. The output is a 2D layer of hidden units. To create multiple output channels, we would repeat this process with multiple kernels, resulting in a 3D tensor of hidden units at hidden layer H_1 .

10.4: Downsampling & Upsampling:

- Earlier, we increased receptive field size by scaling down the representation at each layer using stride=2 convolutions.
- Now we consider methods for scaling down / **downsampling** 2D input representations.
- Also describe methods for scaling them back up / **upsampling**.
↳ useful when the output is an image
- Also describe methods to change the # of channels between layers
↳ useful when recombining representations from 2 branches of a network

Downsampling:

- 3 main approaches to scaling down a 2D representation:
↳ Most common is to scale down both dimensions by a factor of 2.
 - 1) We sample every other position.
↳ When we use a stride of 2, we apply this with the convolution operation.
 - 2) Max Pooling retains the maximum of the 2×2 input values
↳ Induces some invariance to translation. If the input is shifted by 1 pixel, many of these maximum values stay the same.
 - 3) Mean / Average Pooling averages the inputs.
- For all 3, we apply downsampling separately to each channel
↳ so the output is half the width & height but same # of channels.

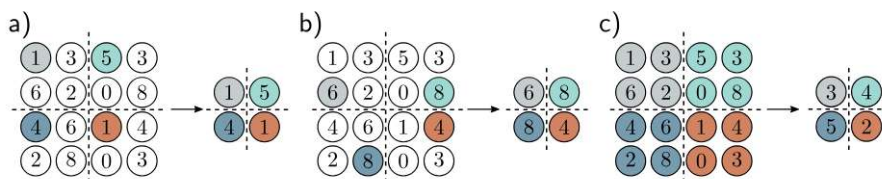


Figure 10.11 Methods for scaling down representation size (downsampling). a)

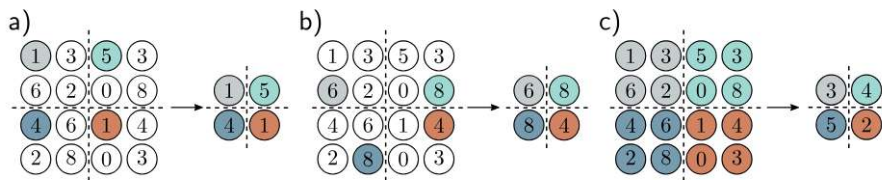


Figure 10.11 Methods for scaling down representation size (downsampling). a) Sub-sampling. The original 4x4 representation (left) is reduced to size 2x2 (right) by retaining every other input. Colors on the left indicate which inputs contribute to the outputs on the right. This is effectively what happens with a kernel of stride two, except that the intermediate values are never computed. b) Max pooling. Each output comprises the maximum value of the corresponding 2x2 block. c) Mean pooling. Each output is the mean of the values in the 2x2 block.

Upsampling:

- The simplest way to scale up a network to double the resolution is to duplicate all the channels at each spatial position 4 times.
- Second method: **Max Unpooling**. Used where we have previously used a max pooling operation for downsampling, we distribute the values to the positions they originated from.
- Third method: Bilinear interpolation to fill in the missing values between the points where we have samples.
- Fourth method: **Transposed convolution**. Roughly analogous to downsampling to downsampling w/ stride of 2.
 - ↳ When downsampling w/ stride=2, there are half as many outputs as inputs; with $K=3$, each output is a weighted sum of the 3 closest inputs.
 - ↳ In transposed convolution, this picture is reversed.
 - ↳ Twice as many outputs as inputs, each input contributes to 3 of the outputs.
 - ↳ The weight matrix is the transpose of the matrix for the downsampling mechanism.

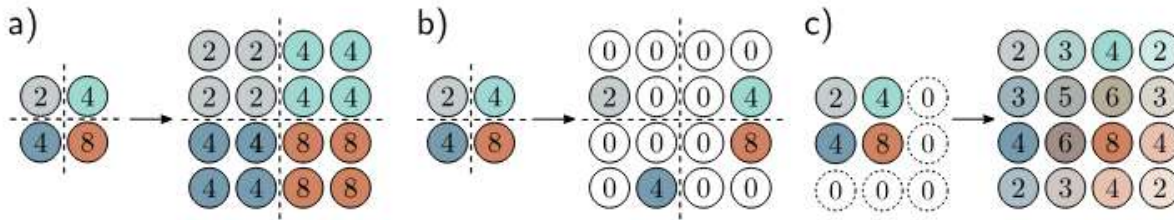


Figure 10.12 Methods for scaling up representation size (upsampling). a) The simplest way to double the size of a 2D layer is to duplicate each input four times. b) In networks where we have previously used a max pooling operation (figure 10.11b), we can redistribute the values to the same positions they originally came from (i.e., where the maxima were). This is known as max unpooling. c) A third option is bilinear interpolation between the input values.

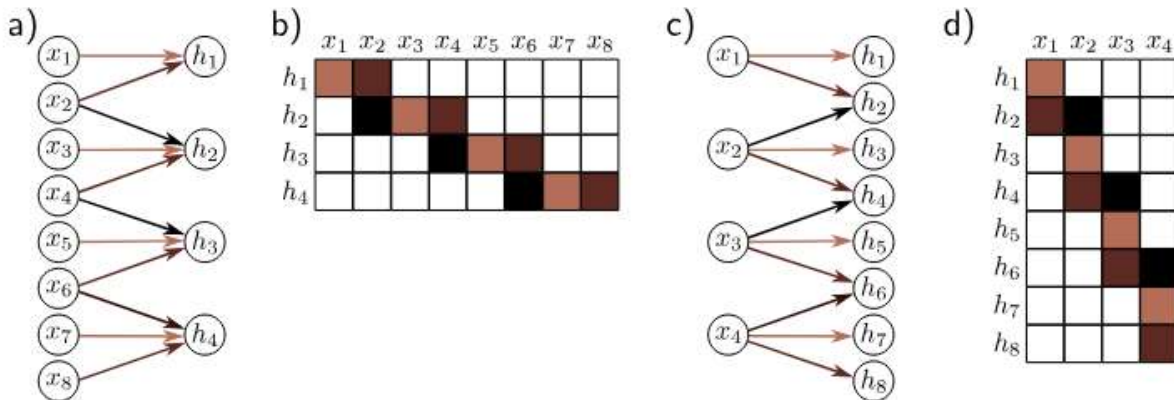
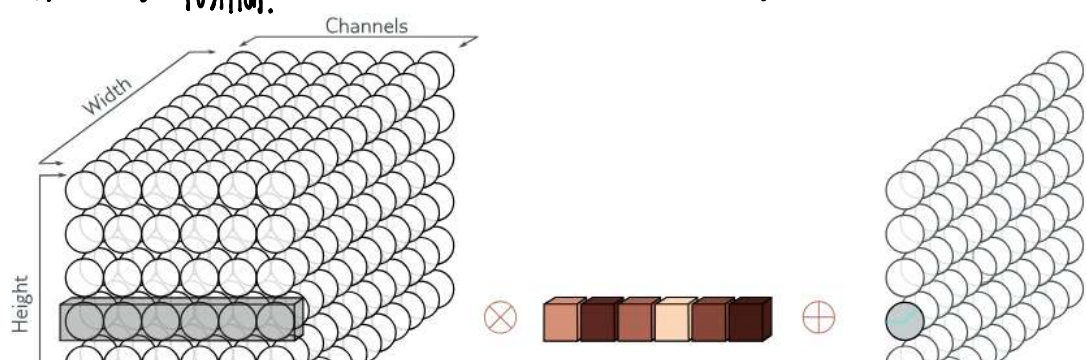


Figure 10.13 Transposed convolution in 1D. a) Downsampling with kernel size three, stride two, and zero-padding. Each output is a weighted sum of three inputs (arrows indicate weights). b) This can be expressed by a weight matrix (same color indicates shared weight). c) In transposed convolution, each input contributes three values to the output layer, which has twice as many outputs as inputs. d) The associated weight matrix is the transpose of that in panel (b).

Changing the Number of Channels:

- Sometimes we want to change the # of channels between hidden layers w/o further spatial Pooling.
 ↳ Usually so we can combine the representation w/ another parallel computation.
- To do this, we apply a convolution w/ kernel size 1.
- Each element of the output layer is computed by taking a weighted sum of all the channels at the same position.



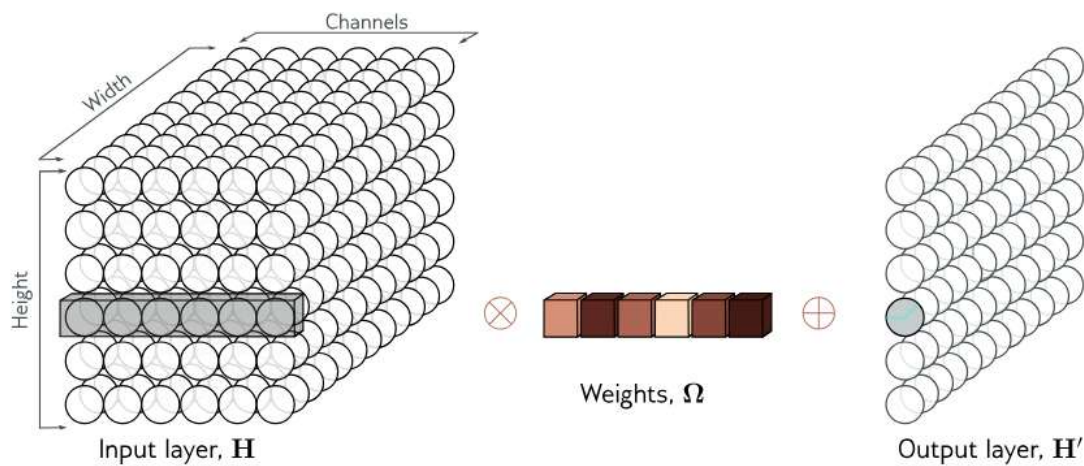


Figure 10.14 1×1 convolution. To change the number of channels without spatial pooling, we apply a 1×1 kernel. Each output channel is computed by taking a weighted sum of all of the channels at the same position, adding a bias, and passing through an activation function. Multiple output channels are created by repeating this operation with different weights and biases.

- Can repeat this multiple times w/ different weights to generate as many output channels as needed.
 - ↳ The associated convolution weights have size $1 \times 1 \times C_i \times C_o$
 - ↳ Known as a 1×1 convolution.

• Combined w/ a bias + activation function, its equivalent to running the same fully connected network on input channels at every position.

10.5: Applications:

Image classification:

- Most pioneering work on DL for computer vision focused on image classification using the ImageNet dataset.
 - ↳ Contains 1,281,167 training images, 50K validation images, 100K test images
 - ↳ Every image is labeled as belonging to 1 of 1,000 possible categories.
- Most methods reshape the input images to a standard size.
 - ↳ Typically x is reshaped to 224×224 RGB image
 - ↳ Output is a probability distribution over the 1,000 classes.
- Challenging task due to the large # of classes, with large variation in between.



Figure 10.15 Example ImageNet classification images. The model aims to assign an input image to one of 1000 classes. This task is challenging because the images vary widely along different attributes (columns). These include rigidity (monkey < canoe), number of instances in image (lizard < strawberry), clutter (compass < steel drum), size (candle < spiderweb), texture (screwdriver < leopard), distinctiveness of color (mug < red wine), and distinctiveness of shape (headland < bell). Adapted from Russakovsky et al. (2015).

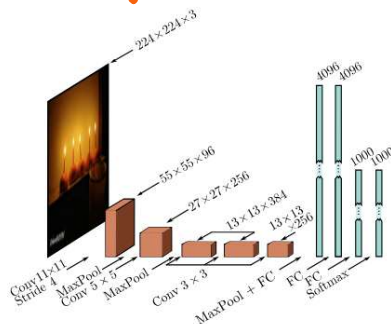
- In 2011, before deep nets were applied, state of the art methods classified the test images w/ ~25% errors for the correct class being in the top-5 suggestions.

- In 2012, AlexNet was the first convolutional network to perform well on the task.

- ↳ Has 8 hidden layers w/ ReLU activation functions

- ↳ First 5 are convolutional, the last are fully connected

Figure 10.16 AlexNet (Krizhevsky et al., 2012). The network maps a 224×224 color image to a 1000-dimensional vector representing class probabilities. The network first convolves with 11×11 kernels and stride 4 to create 96 channels. It decreases the resolution again using a max pool operation and applies a 5×5 convolutional layer. Another max pooling layer follows, and three 3×3 convolutional layers are applied. After a final max pooling operation, the result is vectorized and passed through three fully connected (FC) layers and finally the softmax layer.



- Had ~60 million Parameters, most came from the fully connected layers.

- Dataset size was augmented by a factor of 2048 using spatial transformations ; modification of input intensities.

- At test time, 5 different cropped ; mirrored versions of the image were run through the network, ; their predictions were averaged.

- Used SGD w/ momentum coefficient of 0.9 ; a batch size of 128. Dropout was applied in the fully connected layers ; L2 regularization was used.

- Achieved 16.4% top-5 error ; 38.1% top-1 error.

- ↳ Very big leap!

- The VGG network also targeted image classification on ImageNet. Achieved considerably better performance.

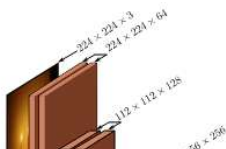
- ↳ 6.8% top-5 error, 23.7% top-1 error.

- ↳ Composed of interspersed convolutional ; max pooling layers

- ↳ The spatial size of the representation gradually decreases, but the # of channels increases.

- ↳ Followed by 3 FC layers.

- ↳ Trained using data augmentation, weight decay, ; dropout.



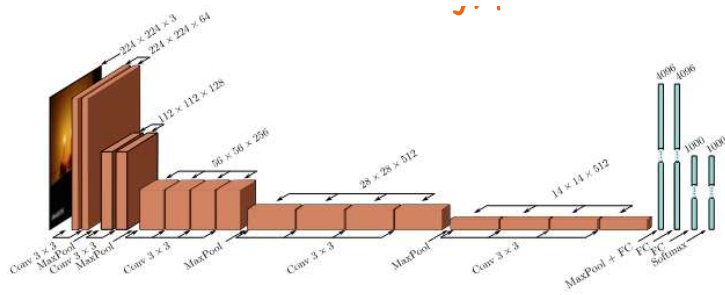


Figure 10.17 VGG network (Simonyan & Zisserman, 2014) depicted at the same scale as AlexNet (see figure 10.16). This network consists of a series of convolutional layers and max pooling operations, in which the spatial scale of the representation gradually decreases, but the number of channels gradually increases. The hidden layer after the last convolutional operation is resized to a 1D vector and three fully connected layers follow. The network outputs 1000 activations corresponding to the class labels that are passed through a softmax function to create class probabilities.

- The most important change between VGG & AlexNet was the depth of the network.
 - ↳ VGG used 14 hidden layers & 144 million parameters.
 - ↳ General trend: For several years, to improve performance, increase depth.
 - ↳ VGG was evidence that depth was important.

Object Detection:

- Goal is to identify & localize multiple objects within the image.
- An early method based on CNNs is YOLO
 - ↳ Input was a 448x448 RGB image
 - ↳ Passed through 24 convolutional layers that gradually decrease the representation size w/ max pooling while increasing the # of channels.
 - ↳ Final convolutional layer is size 7x7 & has 1024 channels.
 - ↳ Reshaped to a vector, & a fully connected layer maps it to 4096 values.
 - ↳ One more fully connected layer maps the representation to the output.
- The output values encode which class is present at each of a 7x7 grid of locations.
 - ↳ For each location the output values also encode a fixed # of bounding boxes.
 - ↳ 5 Params define each box: x-, y- Position of the center, height & width of the box, & the confidence of the prediction.
 - ↳ The confidence estimates the overlap between the predicted & ground truth bounding boxes.
 - ↳ Trained using Momentum, weight decay, dropout, & data augmentation.
 - ↳ Transfer learning is employed; the network is initially trained on the ImageNet classification task, then fine-tuned for object detection.

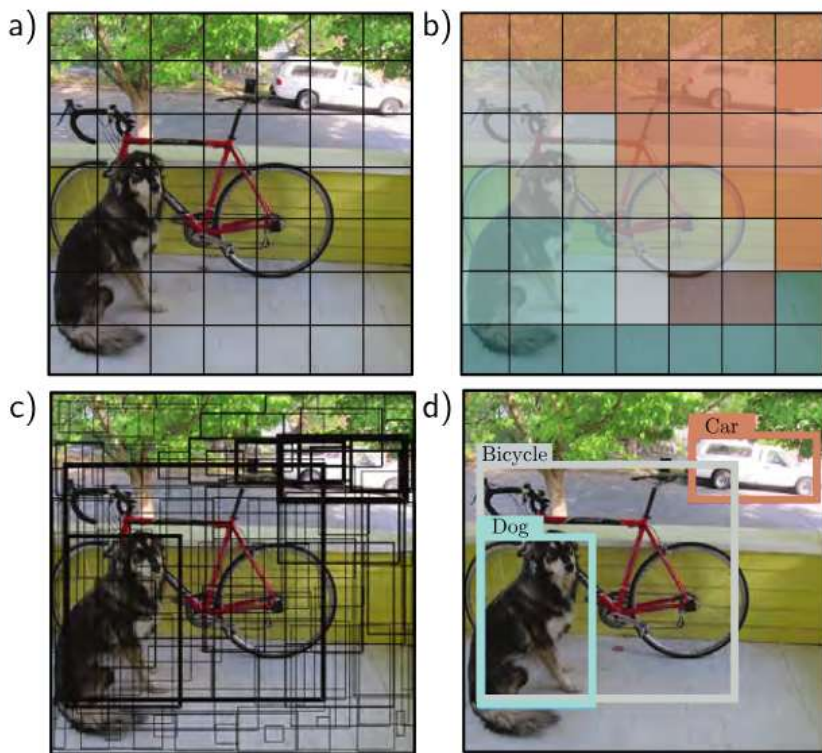


Figure 10.18 YOLO object detection. a) The input image is reshaped to 448×448 and divided into a regular 7×7 grid. b) The system predicts the most likely class at each grid cell. c) It also predicts two bounding boxes per cell, and a confidence value (represented by thickness of line). d) During inference, the most likely bounding boxes are retained, and boxes with lower confidence values that belong to the same object are suppressed. Adapted from Redmon et al. (2016).

- After the network is run, a heuristic process is used to remove rectangles w/ low confidence ; to suppress predicted bounding boxes that correspond to the same object so only the most confident one is kept.

Semantic Segmentation:

- Goal is to assign a label to each pixel according to the object it belongs to, or no label if it doesn't correspond to anything in the training set.
- One early network for it:
 - ↳ Input is 224×224 RGB image.
 - ↳ Output is a $224 \times 224 \times 21$ array that contains the probability of each of 21 possible classes at each input position.
 - ↳ First part of the network is a smaller version of VGG that has 13 conv. layers, ; downsizes the representation to 19×19 .
 - ↳ Followed by a maxpool, ; 2 fully connected layers that map to 2 1D representations of size 4096.
 - ↳ These layers don't represent spatial position, but combine information from across the whole image.
 - ↳ Architecture diverges from VGG here; another FC layer reconstitutes the representation into 7×7 spatial positions ; 512 channels.
 - ↳ Followed by a series of max unpooling layers ; deconvolutional layers.
 - * Transposed convolutions but in 2D ; w/o upsampling.
 - ↳ A 1×1 convolution creates 21 channels representation possible classes. ; a softmax operation at each spatial position to get a probability distribution.

* Transposed Convolutions but in 2D : w/o upsampling.

↳ A 1×1 convolution creates 21 channels representing possible classes, & a softmax operation at each spatial position to map activations to class probabilities.

- The downsampling side of the network is referred to as the **encoder**.

- The Upsampling side is referred to as the **decoder**.

- Networks of this type are called **encoder-decoder networks** [or **hourglass networks**, due to the shape]

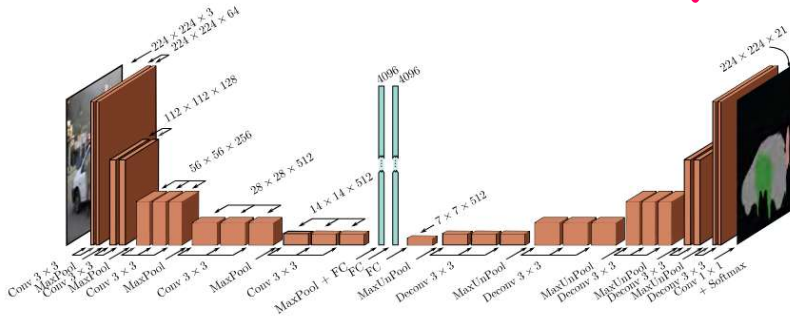


Figure 10.19 Semantic segmentation network of Noh et al. (2015). The input is a 224×224 image, which is passed through a version of the VGG network and eventually transformed into a representation of size 4096 using a fully connected layer. This contains information about the entire image. This is then reformed into a representation of size 7×7 using another fully connected layer, and the image is upsampled and deconvolved (transposed convolutions without upsampling) in a mirror image of the VGG network. The output is a $224 \times 224 \times 21$ representation that gives the output probabilities for the 21 classes at each position.

- Final Segmentation is generated using a heuristic method that greedily searches for the class that is most represented in its region, taking into account the probabilities but also encouraging connectedness.

→ Then the next most-represented class is added where it dominates at the remaining unlabeled pixels.

↳ Loops until there is insufficient evidence to add more.

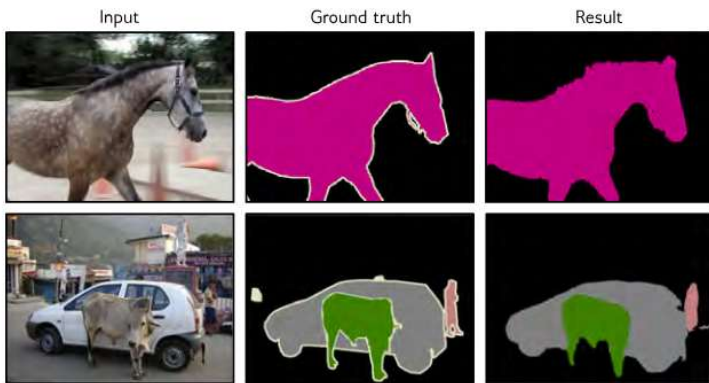


Figure 10.20 Semantic segmentation results. The final result is created from the 21 probability maps by greedily selecting the best class and using a heuristic method to find a sensible binary map based on the probabilities and their spatial proximity. If there is enough evidence, subsequent classes are added, and their segmentation maps are combined. Adapted from Noh et al. (2015).

10.6: Summary:

- In Convolutional layers, each hidden unit is computed by taking a weighted sum of the nearby inputs, adding a bias, applying an activation function.

- The weights + bias are the same at every spatial position; far fewer Params than a FC network, Params don't increase w/ input image size.

- The weights + bias are the same at every spatial position; for fewer Params than a FC network, Params don't increase w/ input image size.
- To ensure information isn't lost, the operation is repeated w/ different weights; biases to make Multiple channels at each spatial position.
- Typical Convolutional Networks consist of convolutional layers interspersed with layers that downsample by a factor of 2.
 - ↳ As a data example passes through the network, spatial dimensions usually decrease by factors of 2, ; Channels increase by a factor of 2.
 - ↳ At the end of the network, there are typically 1 or more FC layers that integrate information from across the entire input ; create the desired output.
 - ↳ If the output is an image, a mirrored decoder upsamples back to the original size.
- The translational equivariance of CNNs imposes a useful inductive bias that increases performance for image-based tasks relative to FC networks.
- Can be applied to:
 - ↳ Image Classification
 - ↳ Object Detection
 - ↳ Semantic Segmentation
 - ↳ ; More...
- We saw in image classification that performance improves as the network becomes deeper.
 - ↳ Further experiments show that increasing network depth indefinitely doesn't continue to help!
 - ↳ After a certain depth, the network becomes difficult to train.
 - ↳ This motivates residual connections.